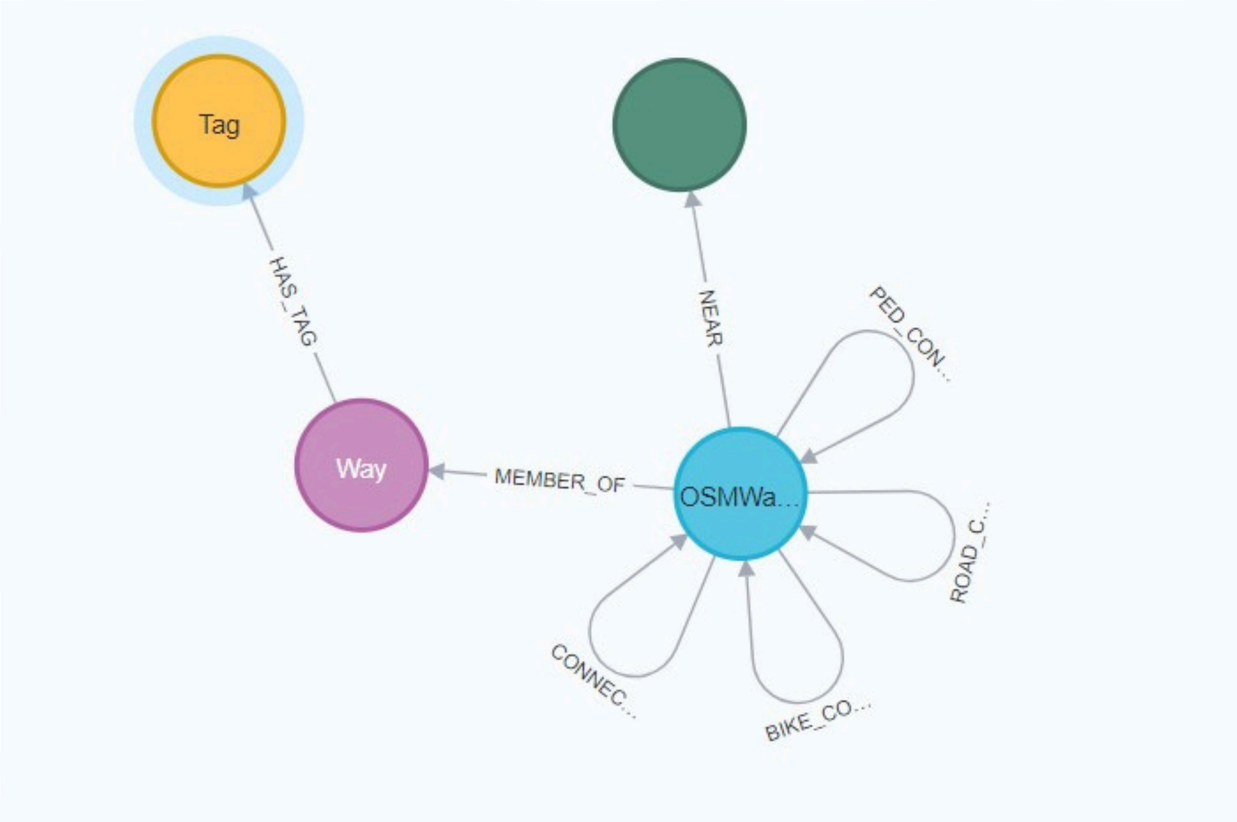
BOLOGNA CITY MAP

costruzione e analisi di una rappresentazione a grafo della città di Bologna utilizzando NEO4J

Singh Damneet Gill N. matr. 207056

GRAPH STRUCTURE



NODI:

- OSMWAYNODE
- WAY
- TAG
- POI

RELAZIONI:

- CONNECTED_TO
- PED_CONNECTED_TO
- BIKE_CONNECTED_TO
- MEMBER_OF
- HAS_TAG
- NEAR

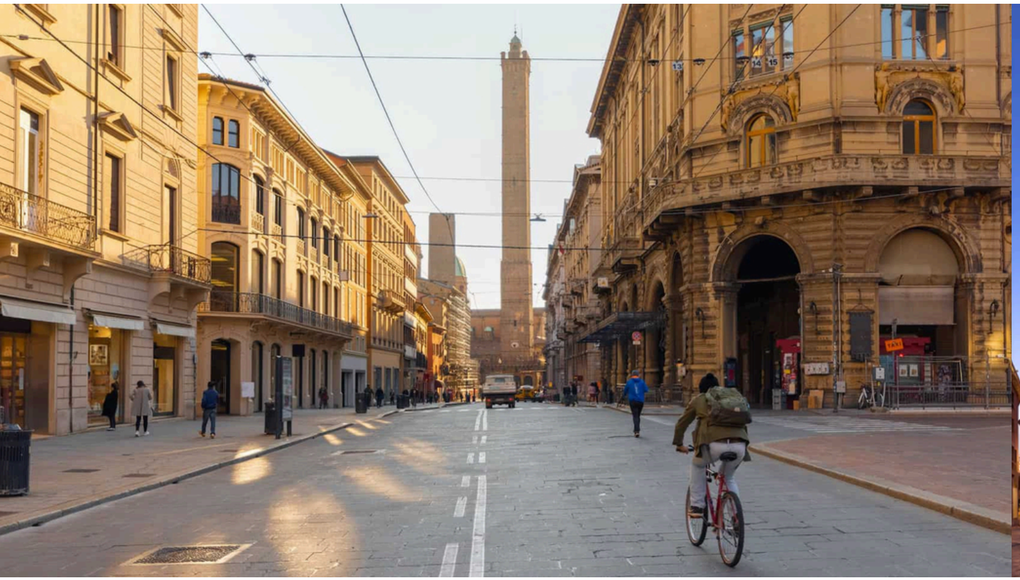
PROPRIETA’


```
{
  "identity": 0,
  "labels": [
    "OSMWayNode"
  ],
  "properties": {
    "elev": 43.97637176513672,
    "x": 1250108.633918686,
    "y": 5548182.174330614,
    "id": "0"
  },
  "elementId": "4:dc32b2ce-c39e-47d4-86f5-1c2d1d8df6b8:0"
}
```

```
{
  "identity": 39868,
  "labels": [
    "Tag"
  ],
  "properties": {
    "value": "stradale",
    "key": "highway"
  },
  "elementId": "4:57d95bfa-d78a-41e7-8c4a-2e089cd9628c:39868"
}
```

```
{
  "identity": 24150,
  "labels": [
    "Way"
  ],
  "properties": {
    "delta_h": 0.4099884033203125,
    "modes": [
      "stradale"
    ],
    "slope_pct": 0.7106679797010852,
    "length": 57.690569299711264,
    "id": "way_0"
  },
  "elementId": "4:dc32b2ce-c39e-47d4-86f5-1c2d1d8df6b8:24150"
}
```

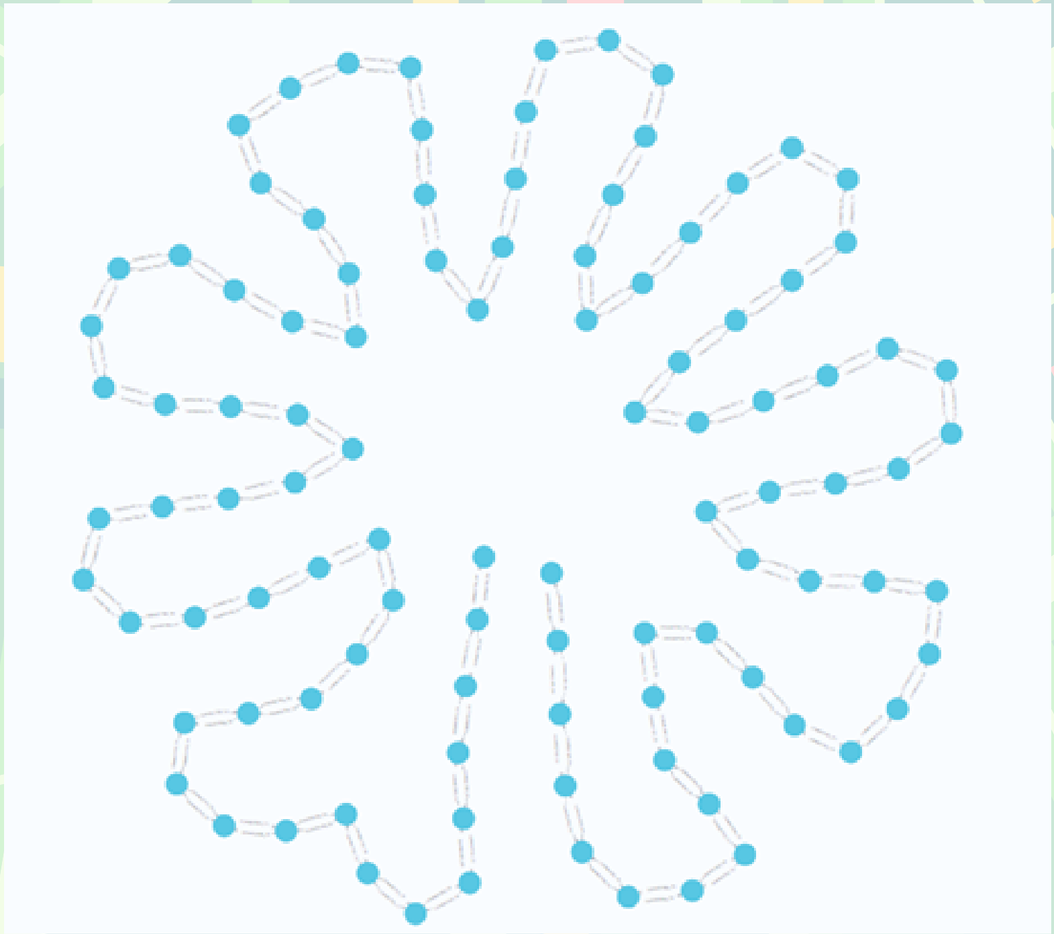
```
{
  "identity": 62442,
  "labels": [
    "POI"
  ],
  "properties": {
    "subtype": "park",
    "x": 1268212.7220387093,
    "y": 5545229.223542962,
    "id": "park_0"
  },
  "elementId": "4:dc32b2ce-c39e-47d4-86f5-1c2d1d8df6b8:62442"
}
```



ROUTE QUERIES

Dopo la costruzione del grafo urbano, Neo4j e Graph Data Science consentono di eseguire analisi di pathfinding per simulare scenari basati sulle esigenze di diversi utenti.

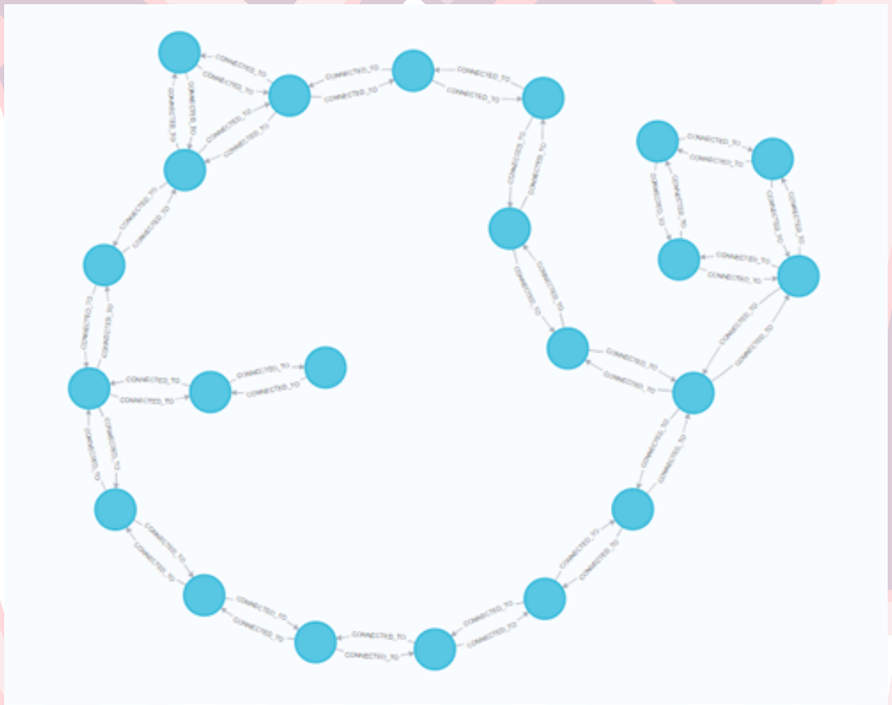
Dopo aver ritornato due nodi random, si trova il percorso più breve tra i due



```
MATCH (n:OSMWayNode)
RETURN n
ORDER BY rand()
LIMIT 2

MATCH (start:OSMWayNode {id: '15420'}),
(end:OSMWayNode {id: '1973'})
MATCH p = shortestPath((start)-
[:CONNECTED_TO*..100]-(end))
RETURN p
```

Calcola un percorso chiuso di una determinata lunghezza a partire da un punto di partenza

```
MATCH (start:OSMWayNode {id:'1973'})
MATCH p = (start)-[:CONNECTED_TO*..30]->(start)
WITH p,
reduce(total = 0, r IN relationships(p) | total + r.length) AS distance
WHERE distance > 4900 AND distance < 5100
RETURN distance, p
LIMIT 1
```



4977.001987265839


```
{
  "start": {
    "identity": 1973,
    "labels": [
      "OSMWayNode"
    ],
    "properties": {
      "elev": 46.75194549560547,
      "x": 1255945.4075656277,
      "y": 5544242.384983978,
      "id": "1973"
```

Calcola un percorso pedonale di una determinata lunghezza a partire da un punto di partenza

```
MATCH (start:OSMWayNode {id:'4705'})
MATCH p = (start)-[:PED_CONNECTED_TO*..30]->()
WITH p,
reduce(total = 0, r IN relationships(p) | total + r.length) AS distance
WHERE distance > 500 and distance < 600
RETURN distance, p
LIMIT 1
```

distance	p
531.4095459447623	{ "start": { "identity": 4705, "labels": ["OSMWayNode"],

Ritorna il percorso più corto tra due punti in cui si considera la pendenza media dei tratti di una certa lunghezza

	id_destinazione	metri	pendenza_accumulata	pendenza_media_percentuale	percorso_nodi
1	"868"	523.6105814584776	0.6251348959429396	0.11938927861268077	["851", "860", "862", "868"]

```
CALL gds.graph.project(
'slopeGraph',
'OSMWayNode',
{
PED_CONNECTED_TO: {
orientation: 'UNDIRECTED',
properties: {
slope_pct: { property: 'slope_pct'},
length: { property: 'length'} }}});
```

```
MATCH (start:OSMWayNode {id: '85l'})

CALL gds.allShortestPaths.dijkstra.stream('slopeGraph', {
sourceNode: start,
relationshipWeightProperty: 'slope_pct'
})
YIELD targetNode, totalCost, nodeIds
WITH gds.util.asNode(targetNode) AS destination,
totalCost AS pendenzaTotale,
nodeIds
WHERE destination.id <> '85l'

UNWIND range(0, size(nodeIds)-2) AS i
WITH destination, pendenzaTotale, nodeIds, nodeIds[i] AS idI, nodeIds[i+1] AS id2
MATCH (nI)-[r:PED_CONNECTED_TO]-(n2)
WHERE id(nI) = idI AND id(n2) = id2
WITH destination,
pendenzaTotale,
sum(r.length) AS lunghezzaReale,
[nodeId IN nodeIds | gds.util.asNode(nodeId).id] AS percorso_nodi

WHERE lunghezzaReale < 600 AND lunghezzaReale > 300

RETURN destination.id AS id_destinazione,
lunghezzaReale AS metri,
pendenzaTotale AS pendenza_accumulata,
(pendenzaTotale / lunghezzaReale) * 100 AS pendenza_media_percentuale,
percorso_nodi
ORDER BY pendenza_media_percentuale ASC
LIMIT 1;
```

Trovare il parco più vicino e per quello trovare il percorso più accessibile e breve

	poiName	totalCost
1	"park_788"	140.5482889694528

```
MATCH ()-[r:PED_CONNECTED_TO]->()
SET r.cost = r.length * (1 + abs(r.slope_pct)/10);

MATCH ()-[r:NEAR]->()
SET r.cost = 1

CALL {
  MATCH (source:OSMWayNode)-[r:PED_CONNECTED_TO]-(target:OSMWayNode)
  RETURN source, target, type(r) AS relType, r.cost AS cost
  UNION
  MATCH (source:OSMWayNode)-[r:NEAR]->(target:POI)
  RETURN source, target, type(r) AS relType, r.cost AS cost
}
WITH gds.graph.project(
  'pedestrianGraph',
  source,
  target,
  {
    sourceNodeLabels: labels(source),
    targetNodeLabels: labels(target),
    relationshipType: relType,
    relationshipProperties: { cost: cost }
  }
) AS g
RETURN g.nodeCount AS nodeCount, g.relationshipCount AS relationshipCount

MATCH (start:OSMWayNode {id: '136'})
CALL gds.allShortestPaths.dijkstra.stream('pedestrianGraph', {
  sourceNode: start,
  relationshipWeightProperty: 'cost'
})
YIELD targetNode, totalCost
WHERE 'POI' IN labels(gds.util.asNode(targetNode))
RETURN gds.util.asNode(targetNode).id AS poiName, totalCost
ORDER BY totalCost ASC
LIMIT 1
```

```
MATCH (n1:OSMWayNode)-[r:CONNECTED_TO]->(n2:OSMWayNode)
OPTIONAL MATCH (n1)-[:MEMBER_OF]->(w:Way)-[:HAS_TAG]->(t:Tag {value: 'portico'})
WHERE (n2)-[:MEMBER_OF]->(w)
SET r.is_covered = CASE WHEN t IS NOT NULL THEN 1 ELSE 0 END,
r.weighted_cost = CASE WHEN t IS NOT NULL THEN r.length ELSE r.length * 3 END;

CALL gds.graph.project(
    'coveredGraph',
    'OSMWayNode',
    'CONNECTED_TO',
    {
relationshipProperties: ['length', 'weighted_cost', 'is_covered']
```

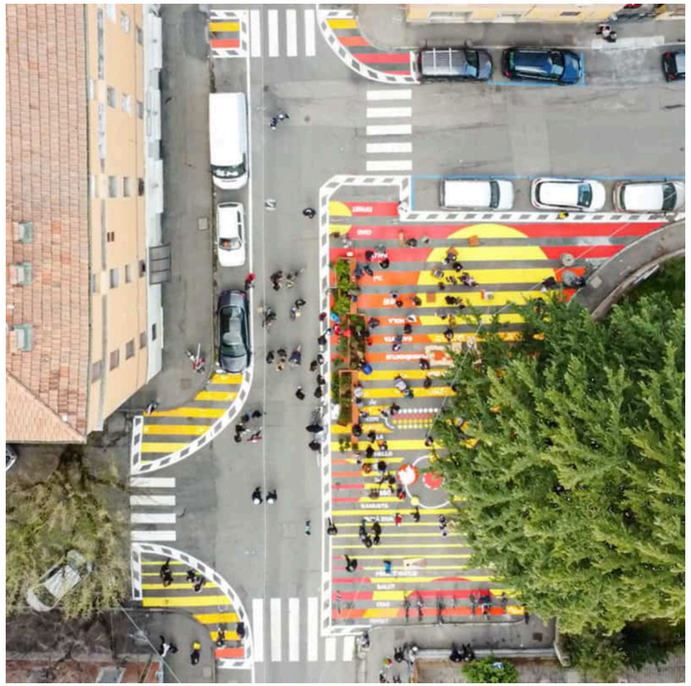
```
MATCH (source:OSMWayNode {id: '1568'}),
      (target:OSMWayNode {id: '1960'})

CALL gds.shortestPath.dijkstra.stream('coveredGraph', {
    sourceNode: source,
    targetNode: target,
relationshipWeightProperty: 'weighted_cost'
})
YIELD nodeIds, totalCost

WITH nodeIds, totalCost, [nodeId IN nodeIds | gds.util.asNode(nodeId).id] AS path_ids

UNWIND range(0, size(nodeIds)-2) AS i
MATCH (n1)-[r:CONNECTED_TO]-(n2)
WHERE id(n1) = nodeIds[i] AND id(n2) = nodeIds[i+1]

WITH path_ids, totalCost, sum(r.is_covered) AS total_covered_arcs

RETURN path_ids,
totalCost AS costo_pesato,
total_covered_arcs
```

Trovare un percorso preferenziale in cui si fa pesare di meno il percorso coperto dal portico

path_ids

["1568", "1569", "1579", "1601", "1607", "1678", "1760", "1777", "1730", "1824", "1878", "1896", "1887", "1906", "1994", "1999", "2159", "2206", "2589", "2894", "2913", "2927", "2928", "2921", "2915", "2886", "28

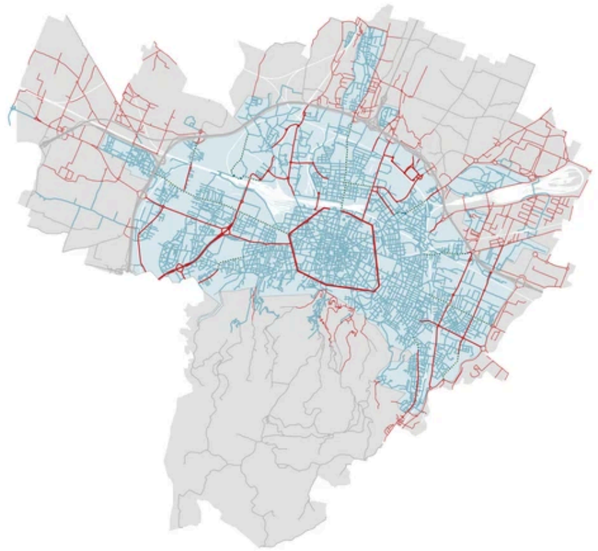
	costo_pesato	total_covered_arcs
, "1997", "1960"]	11467.720626494565	16



URBAN ANALYSIS

L'applicazione di tecniche di Graph Analytics permette di mappare le centralità urbane e di identificare le intersezioni strategiche all'interno del sistema di mobilità


```
CALL gds.graph.project(
'urbanGraph',
'OSMWayNode',
'CONNECTED_TO',
{
relationshipProperties: ['length']
}

);
```



```
CALL gds.louvain.stream('urbanGraph')
YIELD nodeId, communityId
RETURN communityId, count(*)
ORDER BY count(*) DESC;
```

```
CALL gds.louvain.write('urbanGraph', { writeProperty: 'quartiere_id' });
```

```
MATCH (n:OSMWayNode)
WHERE n.quartiere_id IS NOT NULL
RETURN n.quartiere_id AS ID_Community,
avg(n.x) AS Latitudine_Media,
avg(n.y) AS Longitudine_Media,
count(*) AS Numero_Nodi
ORDER BY Numero_Nodi DESC
```

QUARTIERI DI BOLOGNA

	ID_Community	Latitudine_Media	Longitudine_Media	Numero_Nodi
1	7329	1259594.4643183406	5543069.200333396	600
2	18111	1265231.6611964777	5539979.888611555	587
3	16121	1264512.5986333783	5544216.2493706215	567
4	10073	1262379.6527354657	5541286.5312410025	562
5	7649	1261048.7363638736	5542325.715191414	512
6	2588	1257284.854655747	5542231.492857615	504

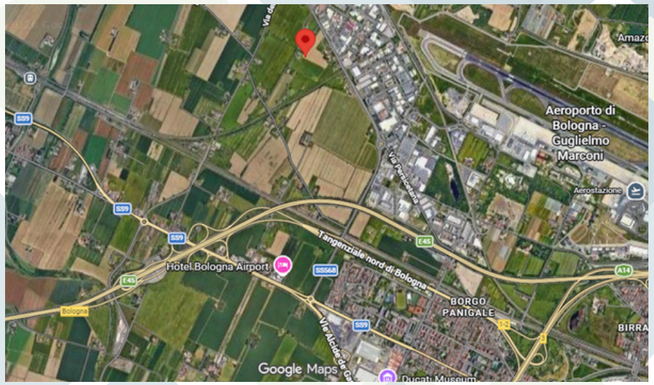
Community	Latitudine	Longitudine	Zona / quartiere di	Nodi
7329	**445.177**	**113.115**	zona **Santa Viola / Barca**	600
18111	**444.979**	**113.621**	zona **San Donato / Pilastro**	587
16121	**445.245**	**113.556**	zona **Corticella / Navile nord**	567
10073	**445.064**	**113.364**	zona **Centro – Bolognina**	562
7649	**445.128**	**113.245**	zona **Saffi / Ospedale Maggiore**	512
2588	**445.120**	**112.910**	zona **Borgo Panigale / aeroporto**	504

Trovare il nodo più connesso


```
{
  "identity": 526,
  "labels": [
    "OSMWayNode"
  ],
  "properties": {
    "x": 1252829.9282097921,
    "y": 5543680.801617972,
    "id": "526",
    "elev": 56.36629867553711
  },
  "elementId": "4:dc32b2ce-c39e-47d4-86f5-1c2d1d8df6b8:526"
}
```



```
{
  "identity": 612,
  "labels": [
    "OSMWayNode"
  ],
  "properties": {
    "x": 1253490.8293390009,
    "y": 5549739.309205114,
    "id": "612",
    "quartiere_id": 469,
    "elev": 37.012298583984375
  },
  "elementId": "4:dc32b2ce-c39e-47d4-86f5-1c2d1d8df6b8:612"
}
```


```
CALL gds.closeness.stream('urbanGraph')
YIELD nodeId, score
RETURN gds.util.asNode(nodeId), score
ORDER BY score DESC
LIMIT 1
```

```
CALL gds.graph.project(
'roadGraph',
'OSMWayNode',
'ROAD_CONNECTED_TO',
{
relationshipProperties: ['length']
}
);
```

```
CALL gds.closeness.stream('roadGraph')
YIELD nodeId, score
RETURN gds.util.asNode(nodeId), score
ORDER BY score DESC
LIMIT 1
```

Trovare gli incroci più connessi

	incrocio	score
1	"4363"	57342777.365801826
2	"5277"	55494143.31022505
3	"5121"	51410797.38049203
4	"5147"	51310793.400054306
5	"5113"	50919510.32275317
6	"5112"	50905526.819343366

	incrocio	score
1	"10568"	53145243.29468349
2	"5583"	50407345.286845475
3	"5058"	48517130.598985896
4	"11650"	48362878.35089885
5	"11769"	48207733.158191614
6	"4789"	47869602.78562825


```
CALL gds.betweenness.stream('urbanGraph')
YIELD nodeId, score
RETURN gds.util.asNode(nodeId).id AS incrocio, score
ORDER BY score DESC
LIMIT 10
```

```
CALL gds.betweenness.stream('roadGraph')
YIELD nodeId, score
RETURN gds.util.asNode(nodeId).id AS incrocio, score
ORDER BY score DESC
LIMIT 10
```

GRAZIE PER
L’ATTENZIONE